

UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Curso de Graduação Tecnológica em Ciência de Dados

SISTEMA DE RECOMENDAÇÃO DE CURSOS ONLINE

Uma abordagem por KNN baseada em conteúdo usando Python

Eduardo Pinheiro Canas

Hugo de Moraes Holzer

Luiz Rodrigo Alves Vergino

Orientadora: Prof.^a Dr.^a Carolina Toledo Ferraz

São Paulo / SP — 2026

SUMÁRIO

1. INTRODUÇÃO	3
1.1 CONTEXTO DO TRABALHO	3
1.2 MOTIVAÇÃO	5
1.3 JUSTIFICATIVA	5
1.4 OBJETIVOS	6
2. REFERENCIAL TEÓRICO	7
3. METODOLOGIA	10
3.1 DEFINIÇÃO DO PROBLEMA E OBJETIVOS	11
3.2 COLETA DOS DADOS	11
3.3 PRÉ-PROCESSAMENTO E LIMPEZA DOS DADOS	12
3.4 DIVISÃO DOS DADOS	12
3.5 SELEÇÃO E IMPLEMENTAÇÃO DO ALGORITMO	13
3.6 TREINAMENTO DO MODELO	14
3.7 FUNÇÃO DE RECOMENDAÇÃO	14
3.8 AVALIAÇÃO DO DESEMPENHO	16
3.9 OTIMIZAÇÃO E AJUSTES	16
3.10 IMPLANTAÇÃO E MONITORAMENTO	17
4. REFERÊNCIAS	18

1. INTRODUÇÃO

1.1 CONTEXTO DO TRABALHO

O crescimento das plataformas de ensino online transformou profundamente o acesso à educação, tornando disponíveis milhares de cursos de instituições ao redor do mundo. A Coursera, uma das maiores plataformas do setor, conta com mais de 7.000 cursos oferecidos por universidades e empresas parceiras, cobrindo áreas que vão de ciência de dados a gestão de negócios. Nesse cenário de vasta informação, identificar o curso mais adequado ao perfil do estudante e aos objetivos de cada um torna-se um desafio crescente — fenômeno conhecido como sobrecarga de escolha (RICCI et al., 2011).

É nesse contexto que sistemas de recomendação assumem papel estratégico: ao filtrar e personalizar o catálogo disponível com base nas características do conteúdo ou no histórico do usuário, essas ferramentas reduzem a fricção entre o aprendiz e a trilha de aprendizado mais relevante para ele. Este Projeto Aplicado propõe o desenvolvimento de um sistema de recomendação de cursos online utilizando o Coursera Courses Dataset 2021, base pública disponível na plataforma Kaggle, no link:

<https://www.kaggle.com/datasets/khusheekapoor/coursera-courses-dataset-2021>

Coursera Courses Dataset 2021

111

Code

Download

Data Card

Code (18)

Discussion (1)

Suggestions (0)

Coursera.csv (5.28 MB)

Detail

Compact

Columns

7 of 7 columns

About this file

Suggest Edits

This table contains information publicly available on the Coursera website. The columns are: Name, University, Difficulty Level, Rating, Link, Description and Skills.

A Course Name	A University	A Difficulty Level	A Course Rating	= Course URL	A Course Description	A Skills
The Name of the Course.	The University or Industry Partner that offers the Course.	The Difficulty Level of the Course. Has 3 values: Beginner, Intermediate, Advanced. Also has Missing Values	Rating on a 5-point scale with minimum step value 0.1. Missing Values represented by Not Calibrated.	The Course URL.	The Description of the Course. Has Missing Values.	The Skill Tags associated with the Course extracted through NLP processes.
3416 unique values	Coursera Project ... 16% University of Illinois... 4% Other (2822) 80%	Beginner 41% Advanced 29% Other (1072) 30%	4.7 21% 4.6 18% Other (2158) 61%	3424 unique values	3397 unique values	3424 unique values

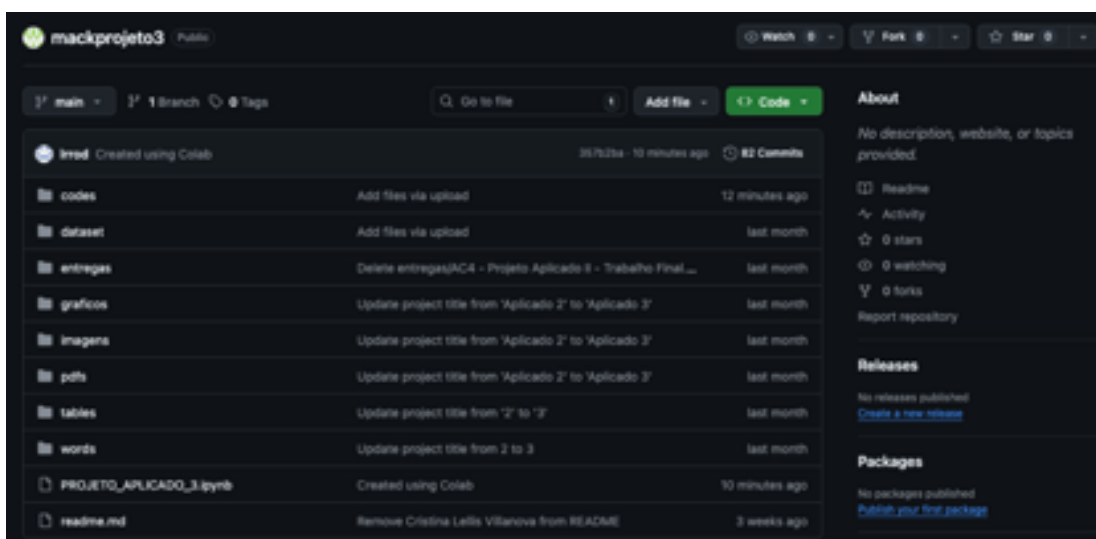
Esta base possui 3.522 cursos com atributos como nome, universidade, nível de dificuldade, nota média, descrição e habilidades desenvolvidas.

A abordagem adotada é a filtragem baseada em conteúdo, com vetorização TF-IDF aplicada sobre os atributos textuais dos cursos e busca de similares pelo algoritmo K-Nearest Neighbors (KNN) com métrica de similaridade de cosseno. Essa escolha é motivada pela ausência de avaliações individuais de usuários na base, característica que inviabiliza a filtragem colaborativa e direciona a solução para a análise dos atributos dos próprios cursos.

O projeto está alinhado aos Objetivos de Desenvolvimento Sustentável 4, **Educação de Qualidade**, e, 8, **Trabalho Decente e Crescimento Econômico**, da Agenda 2030 da ONU. Ao sugerir trilhas de aprendizado personalizadas de forma autônoma, a solução contribui para que pessoas de diferentes perfis socioeconômicos possam identificar oportunidades de qualificação profissional sem depender de orientação educacional especializada, impactando diretamente na democratização do acesso ao conhecimento e no fortalecimento do capital humano em um cenário de transformação digital acelerada.

O projeto está sendo desenvolvido em formato de notebook Python (Google Colab), com código aberto disponível no repositório GitHub:

<https://github.com/canasep/mackprojeto3>



O repositório segue os princípios de reprodutibilidade científica, utilizando exclusivamente dados e ferramentas publicamente acessíveis.

1.2 MOTIVAÇÃO

A motivação em desenvolver este trabalho surge da importância crescente das plataformas de ensino online no cenário educacional atual, especialmente após a aceleração da transformação digital. Entender como os sistemas de recomendação funcionam na prática é uma forma de aplicar os conceitos de Machine Learning estudados ao longo do curso em uma situação real e de alto impacto social.

Existe também um interesse em compreender como diferentes abordagens de recomendação se diferenciam em termos de aplicabilidade e resultados, além é claro, de experimentar o desenvolvimento completo de um pipeline de Machine Learning, desde a coleta e tratamento de dados, até a análise exploratória dos dados e a avaliação quantitativa do modelo.

1.3 JUSTIFICATIVA

O crescimento acelerado das plataformas de ensino online criou um paradoxo educacional: quanto maior o catálogo disponível, maior a dificuldade do usuário em encontrar o curso mais adequado ao seu perfil e objetivos. Nesse contexto, sistemas de recomendação atuam como facilitadores do acesso ao conhecimento, reduzindo a barreira entre o aprendiz e a trilha educacional mais adequada ao seu momento de vida (RICCI et al., 2011).

Do ponto de vista social, a relevância deste projeto se expressa em sua contribuição direta ao ODS 4, Educação de Qualidade, que estabelece como meta assegurar a educação inclusiva, equitativa e de qualidade, promovendo oportunidades de aprendizado ao longo da vida para todos. Ao recomendar cursos alinhados às habilidades e ao nível de experiência do usuário, a solução contribui para que pessoas com diferentes perfis, independentemente de acesso a orientação educacional especializada ou de renda, possam identificar oportunidades de aprendizado de forma autônoma e eficiente.

O projeto também se alinha ao ODS 8, Trabalho Decente e Crescimento Econômico. Em um cenário de transformação digital acelerada, onde novas competências técnicas tornam-se requisitos frequentes no mercado de trabalho, ferramentas que auxiliam na identificação de caminhos de qualificação profissional possuem impacto direto na empregabilidade e no desenvolvimento de carreira, contribuindo para a redução de desigualdades na formação técnica e para o fortalecimento do capital humano.

1.4 OBJETIVOS

O objetivo geral deste projeto é desenvolver, implementar e avaliar um sistema de recomendação de cursos online baseado em filtragem por conteúdo, utilizando o algoritmo K-Nearest Neighbors (KNN) com vetorização TF-IDF aplicado ao Coursera Courses Dataset 2021, documentando todas as etapas do processo em formato científico e alinhando a solução aos ODS 4 e ODS 8 da ONU.

Para atingir esse objetivo, foram definidos os seguintes objetivos específicos:

- Selecionar e carregar a base de dados Coursera Courses Dataset 2021 via API Kaggle.
- Realizar a análise exploratória dos dados (EDA) para identificar distribuições, padrões, inconsistências e características relevantes do dataset.
- Executar a limpeza, o tratamento de valores ausentes e a engenharia do campo features combinado.
- Aplicar a vetorização TF-IDF com configuração adequada (max_features, ngram_range, min_df, stop_words).
- Implementar o modelo KNN com similaridade de cosseno para recomendação baseada em conteúdo.
- Desenvolver função de recomendação com dois modos: por nome de curso e por skill/interesse livre.
- Avaliar o sistema por meio de métricas de cobertura, similaridade, diversidade e qualidade das recomendações.
- Gerar visualizações analíticas: heatmap de similaridade e comparativo de distribuição de dificuldade.

- Documentar a metodologia, os experimentos e os resultados obtidos em formato acadêmico.
- Analisar criticamente os pontos fortes, limitações e propor melhorias futuras para o sistema.

2. REFERENCIAL TEÓRICO

A expansão das plataformas de ensino online gerou um volume de conteúdo que supera a capacidade humana de navegação eficiente. Nesse contexto, sistemas de recomendação emergem como instrumentos capazes de personalizar o acesso ao conhecimento em escala, sugerindo automaticamente os itens mais relevantes para cada perfil de usuário com base em suas características ou histórico de interação (RICCI et al., 2011). Aplicados à educação, esses sistemas têm potencial de reduzir desigualdades no acesso a trilhas de aprendizado de qualidade, contribuindo diretamente para o ODS 4 da ONU, que estabelece a educação inclusiva e equitativa como meta global. Ao orientar o aprendiz de forma autônoma e personalizada, independentemente de sua renda ou acesso a orientação especializada, a tecnologia torna-se veículo de democratização do conhecimento (UNESCO, 2023).

O impacto social dos sistemas de recomendação vai além da conveniência individual: em um cenário de transformação digital acelerada, onde novas competências técnicas são exigidas continuamente pelo mercado de trabalho, ferramentas que auxiliam na identificação de trilhas de qualificação profissional acessíveis possuem efeito direto sobre a empregabilidade e o desenvolvimento econômico, alinhando-se ao ODS 8, que visa o crescimento econômico inclusivo e o trabalho decente para todos (ONU, 2015).

Assim, a tecnologia de recomendação deixa de ser apenas um recurso de engajamento digital para se tornar um instrumento de política social aplicada, capaz de ampliar oportunidades em escala sem custo marginal adicional por usuário atendido.

Entre as abordagens existentes, filtragem colaborativa, filtragem baseada em conteúdo e métodos híbridos, a filtragem por conteúdo destaca-se por sua capacidade de operar sem histórico de interações de outros usuários (LOPS; DE

GEMMIS; SEMERARO, 2011). Essa característica é determinante para o presente projeto: o Coursera Courses Dataset 2021 contém atributos ricos sobre os cursos (nome, descrição, habilidades, nível de dificuldade e universidade ofertante), mas não registra avaliações individuais de usuários. A filtragem baseada em conteúdo representa cada curso como um conjunto de atributos e calcula a similaridade entre itens diretamente a partir dessas representações, sem depender de comportamento coletivo.

Para que a similaridade entre cursos possa ser calculada computacionalmente, é necessário transformar seus atributos textuais em representações numéricas. A técnica TF-IDF (Term Frequency-Inverse Document Frequency), proposta por Salton e Buckley (1988), realiza essa transformação atribuindo pesos a cada termo proporcional à sua frequência no documento e inversamente proporcional à sua frequência no corpus, ou seja, termos técnicos específicos de um curso recebem maior peso do que palavras comuns a todo o catálogo.

O resultado é uma matriz esparsa onde cada curso é representado como um vetor numérico que captura sua identidade temática. Essa representação é fundamental: ela cria o espaço vetorial no qual o algoritmo de busca por similaridade irá operar (MANNING; RAGHAVAN; SCHÜTZE, 2008).

O algoritmo K-Nearest Neighbors (KNN) é especialmente adequado para atuar nesse espaço vetorial gerado pelo TF-IDF. Proposto por Cover e Hart (1967), o KNN é um método baseado em instâncias que, para cada consulta, identifica os K pontos mais próximos no espaço de representação sem necessidade de treinamento supervisionado com rótulos. No contexto de recomendação de cursos, isso significa encontrar os K cursos cujos vetores TF-IDF são mais similares ao curso consultado ou à habilidade informada pelo usuário. A ausência de uma etapa de treinamento com variável-resposta (y) é uma vantagem direta neste projeto, já que não há dados de feedback explícito de usuários disponíveis.

A métrica de similaridade de cosseno é a escolha natural para comparar vetores TF-IDF esparsos. Diferentemente da distância euclidiana, a similaridade de cosseno mede o ângulo entre dois vetores independentemente de sua magnitude,

ou seja, cursos com descrições longas e curtas são comparados em igualdade de condições, desde que compartilhem vocabulário técnico similar (SINGHAL, 2001). Valores próximos de 1 indicam alta proximidade temática; valores próximos de 0 indicam cursos sem relação de conteúdo. Essa propriedade torna o cosseno a métrica de referência para recuperação de informação em espaços esparsos de alta dimensionalidade, como a matriz 3.522×5.000 gerada neste projeto.

A combinação TF-IDF + KNN com similaridade de cosseno forma, portanto, um pipeline coerente e tecnicamente fundamentado: o TF-IDF cria uma representação numérica fiel ao conteúdo de cada curso, e o KNN utiliza essa representação para identificar os vizinhos mais próximos no espaço vetorial resultante. Essa arquitetura é interpretável, computacionalmente eficiente para datasets de dimensão moderada e produz recomendações diretamente justificáveis pelo conteúdo dos cursos, características alinhadas ao princípio de transparência em sistemas de inteligência artificial aplicados ao contexto educacional (EUROPEAN COMMISSION, 2019).

Por fim, a avaliação de sistemas de recomendação baseados em conteúdo, na ausência de feedback explícito de usuários, é realizada por métricas offline que analisam o comportamento do modelo sobre amostras do catálogo (HERLOCKER et al., 2004).

As métricas tradicionais de recuperação de informação: Precision, Recall e F1-Score, requerem rótulos de relevância previamente definidos por usuário, indisponíveis neste dataset. As quatro métricas adotadas neste projeto: cobertura do catálogo, similaridade média, diversidade média e qualidade das recomendações, foram selecionadas por sua adequação à abordagem content-based e por permitirem avaliar dimensões complementares do desempenho do sistema: relevância, alcance, variedade e qualidade percebida das sugestões.

3. METODOLOGIA

Esta seção descreve a metodologia adotada para o desenvolvimento do sistema de recomendação de cursos online, detalhando cada etapa do pipeline desde a definição do problema até a discussão sobre implantação e monitoramento. O esquema geral da metodologia é apresentado na Figura 1; nas subseções seguintes, cada etapa é descrita em detalhe.

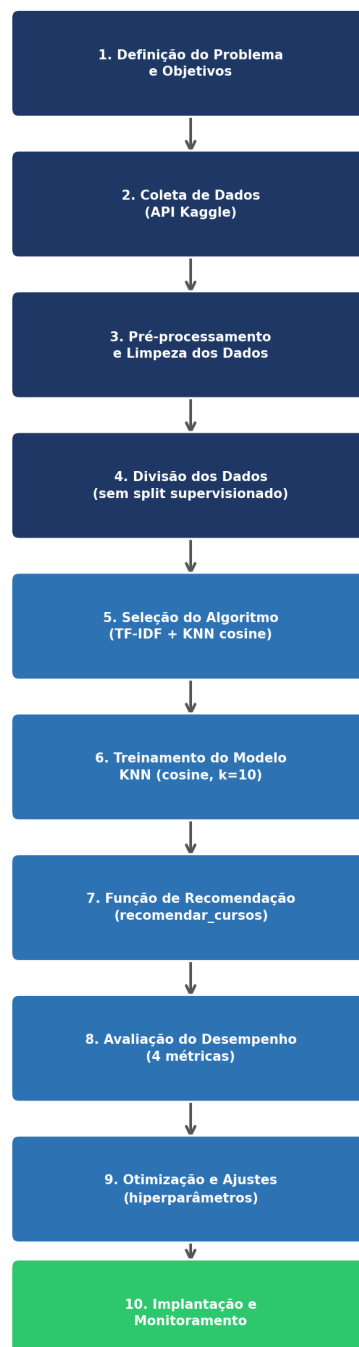


Figura 1 – Fluxograma da metodologia utilizada no projeto

3.1 DEFINIÇÃO DO PROBLEMA E OBJETIVOS

Nesta etapa, definiu-se de forma clara o problema central do projeto: como recomendar cursos online de forma personalizada em um dataset que não possui histórico de interações individuais de usuários. A ausência de dados de comportamento (ratings por usuário, histórico de cliques ou matrículas) inviabiliza a filtragem colaborativa e direciona a solução para a abordagem de filtragem baseada em conteúdo, na qual a similaridade é calculada com base nos atributos dos próprios cursos.

O objetivo geral estabelecido foi desenvolver um sistema capaz de recomendar cursos relevantes a partir de um curso de referência ou de uma consulta livre de habilidade (skill), utilizando vetorização TF-IDF e o algoritmo KNN com similaridade de cosseno. Os objetivos específicos detalham as etapas técnicas e acadêmicas necessárias para atingir esse objetivo, conforme listado na Seção 1.4.

3.2 COLETA DOS DADOS

A coleta de dados foi realizada por meio da API oficial do Kaggle, utilizando o comando `kaggle datasets download` com o identificador do dataset `khusheekapoor/coursera-courses-dataset-2021`. O arquivo CSV resultante foi carregado diretamente em um DataFrame Pandas com a instrução `pd.read_csv('Coursera.csv')`.

O dataset Coursera Courses Dataset 2021 contém 3.522 registros e 7 colunas: Course Name (nome do curso), University (instituição ofertante), Difficulty Level (nível de dificuldade), Course Rating (nota média), Course URL (endereço de acesso), Course Description (descrição textual) e Skills (habilidades desenvolvidas). Os nomes das colunas foram padronizados para letras minúsculas e renomeados para facilitar o acesso programático: `name`, `university`, `difficulty`, `rating`, `url`, `description` e `skills`.

A escolha por este dataset foi motivada por sua representatividade: 3.522 cursos de mais de 30 instituições parceiras, pela riqueza dos campos textuais disponíveis para vetorização e pela disponibilidade pública e gratuita na plataforma Kaggle, alinhando-se ao princípio de reprodutibilidade científica.

3.3 PRÉ-PROCESSAMENTO E LIMPEZA DOS DADOS

O pré-processamento foi estruturado em duas frentes: limpeza textual e tratamento de valores ausentes.

Para a limpeza textual, foi implementada a função `limpar_texto()`, que executa três operações sequenciais: (1) conversão de todo o texto para letras minúsculas, eliminando sensibilidade a maiúsculas; (2) remoção de caracteres especiais e pontuação por meio da expressão regular `[^a-z0-9\s]`, mantendo apenas letras, dígitos e espaços; (3) normalização de múltiplos espaços em um único espaço com `strip()`. Registros nulos são convertidos para strings vazias antes do processamento.

```
def limpar_texto(texto):
    if pd.isna(texto): return ''
    texto = str(texto).lower()
    texto = re.sub(r'[^a-z0-9\s]', ' ', texto)
    texto = re.sub(r'\s+', ' ', texto).strip()
    return texto
```

Para o tratamento de valores ausentes, os campos textuais `skills` e `description` foram preenchidos com strings vazias (`''`). O campo `difficulty` foi imputado com o valor `'Mixed'` como padrão, representando a categoria mais neutra do dataset. O campo `rating` foi preenchido com a mediana das avaliações disponíveis — estratégia mais robusta à assimetria da distribuição e à presença de outliers do que a média aritmética. Não foram identificados registros duplicados, pois cada curso possui uma URL única na plataforma Coursera.

3.4 DIVISÃO DOS DADOS

Em projetos de Machine Learning supervisionado, a etapa de divisão dos dados em conjuntos de treino e teste é fundamental para estimar a capacidade de generalização do modelo. No entanto, a abordagem de filtragem baseada em conteúdo adotada neste projeto opera de forma diferente: o modelo KNN não aprende a partir de rótulos supervisionados (`y`), mas constrói uma estrutura de busca sobre a representação vetorial dos próprios cursos.

Dessa forma, não existe uma separação clássica de dados de treino e teste no pipeline implementado. O modelo é ajustado (fit) sobre a matriz TF-IDF completa dos 3.522 cursos e, em seguida, utiliza essa mesma estrutura para responder a consultas de similaridade. A avaliação de desempenho é realizada indiretamente sobre uma amostra aleatória de 200 cursos, calculando métricas de cobertura, similaridade, diversidade e qualidade das recomendações geradas, sem a necessidade de um conjunto de "respostas corretas" pré-rotuladas.

Esta característica é comum e esperada em sistemas de recomendação baseados em conteúdo sem dados de feedback explícito de usuários, conforme descrito por Herlocker et al. (2004). A ausência de split não é uma limitação técnica do projeto, mas uma consequência direta do tipo de dado disponível e da abordagem metodológica adotada.

3.5 SELEÇÃO E IMPLEMENTAÇÃO DO ALGORITMO

A seleção do algoritmo K-Nearest Neighbors (KNN) com similaridade de cosseno foi fundamentada em três critérios principais: adequação ao tipo de dado, interpretabilidade dos resultados e eficiência computacional para o volume do dataset.

A similaridade de cosseno é a métrica mais indicada para representações TF-IDF esparsas porque é invariante à norma dos vetores, ou seja, não é afetada pelo comprimento da descrição dos cursos, mas apenas pela composição temática (ângulo entre vetores). Cursos com descrições longas e cursos com descrições curtas são comparados em igualdade de condições, desde que compartilhem vocabulário similar (SINGHAL, 2001).

O algoritmo de busca brute-force (algorithm='brute') foi selecionado por garantir exatidão na busca dos vizinhos mais próximos e por ser adequado para matrizes esparsas de dimensão moderada como a utilizada (3.522×5.000 termos). Alternativas como `ball_tree` e `kd_tree` não suportam métricas de cosseno diretamente em matrizes esparsas no Scikit-learn.

Foram consideradas alternativas como Matrix Factorization (SVD) e abordagens neurais, porém descartadas por exigirem dados de interação usuário-item, ausentes no dataset selecionado.

3.6 TREINAMENTO DO MODELO

O treinamento do modelo consistiu em duas etapas sequenciais (5 e 6 do arquivo Projeto_Aplicado_3 no Colab): vetorização TF-IDF e ajuste do modelo KNN.

Na vetorização, o `TfidfVectorizer` foi instanciado com os parâmetros `max_features=5000`, `gram_range=(1,2)`, `min_df=2` e `stop_words='english'`, e ajustado sobre a coluna `features` do `DataFrame` completo por meio do método `fit_transform()`. O resultado é uma matriz esparsa de dimensão 3.522×5.000 , onde cada linha representa um curso como vetor de pesos TF-IDF.

```
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2),
                        min_df=2, stop_words='english')
tfidf_matrix = tfidf.fit_transform(df['features'])

model_knn = NearestNeighbors(metric='cosine', algorithm='brute',
                             n_neighbors=11, n_jobs=-1)
model_knn.fit(tfidf_matrix)
```

Em seguida, o modelo `NearestNeighbors` foi instanciado com os parâmetros `metric='cosine'`, `algorithm='brute'`, `n_neighbors=11` e `n_jobs=-1`, e ajustado sobre a matriz TF-IDF completa. O parâmetro `n_neighbors=11` é definido como $K+1$, onde $K=10$ é o número de recomendações desejadas, pois o modelo retorna o próprio curso de referência como o vizinho mais próximo (distância zero), que é descartado na função de recomendação.

3.7 FUNÇÃO DE RECOMENDAÇÃO

A função `recomendar_cursos()` encapsula toda a lógica de inferência do sistema, traduzindo o modelo treinado em uma ferramenta de uso direto. Ela recebe como parâmetros o texto de consulta (nome de um curso ou skill de interesse livre), o número de recomendações desejadas (padrão: 10) e um indicador de modo (por_skill=False para busca por curso, por_skill=True para busca livre). O retorno é

um DataFrame ordenado por similaridade de cosseno decrescente, contendo nome, universidade, dificuldade, nota e percentual de similaridade de cada curso recomendado.

A função suporta dois modos de operação complementares. No modo por curso de referência (padrão), o nome informado é localizado no DataFrame e o vetor TF-IDF pré-calculado daquele índice é utilizado diretamente como ponto de consulta: sem necessidade de nova vetorização. No modo por skill livre, o texto fornecido pelo usuário é pré-processado pela função `limpar_texto()` e vetorizado em tempo real pelo mesmo `TfidfVectorizer` ajustado no treinamento, gerando um vetor de consulta no mesmo espaço dimensional da matriz principal. Em ambos os casos, `model_knn.kneighbors()` recupera os K+1 vizinhos mais próximos; o próprio item de referência (índice 0, distância zero) é descartado do resultado final.

```
def recomendar_cursos(query, n=10, por_skill=False):
    if por_skill:
        query_vec = tfidf.transform([limpar_texto(query)])
    else:
        idx = df[df['name'].str.lower() == query.lower()].index[0]
        query_vec = tfidf_matrix[idx]
    distances, indices = model_knn.kneighbors(query_vec, n_neighbors=n+1)
    recomendados = df.iloc[indices[0][1:]].copy()
    recomendados['similaridade'] = 1 - distances[0][1:]
    return
    recomendados[['name', 'university', 'difficulty', 'rating', 'similaridade']]
```

Três cenários de teste foram executados para validar o funcionamento da função: (1) busca por curso de referência com nível Beginner ("Machine Learning by Stanford University"), verificando que os vizinhos retornados compartilham vocabulário técnico de aprendizado de máquina; (2) busca por skill livre com o termo "data science", avaliando a capacidade de vetorização em tempo real sobre texto não presente no índice; e (3) busca por skill livre com "project management", validando o comportamento em um domínio temático distinto. Em todos os cenários, a função retornou 10 recomendações ordenadas por similaridade, confirmando o funcionamento correto dos dois modos de operação.

3.8 AVALIAÇÃO DO DESEMPENHO

A avaliação do desempenho foi realizada sobre uma amostra aleatória de 200 cursos selecionados com `numpy.random.choice()`, calculando quatro métricas para cada consulta e resumizando os resultados com média e desvio padrão.

A Cobertura do catálogo foi calculada como o percentual de cursos distintos presentes nas listas de recomendação geradas para a amostra em relação ao total de 3.522 cursos. A Similaridade média foi calculada como a média das similaridades de cosseno ($1 - \text{distância}$) entre o curso de referência e seus 10 vizinhos mais próximos. A Diversidade média foi calculada como a distância média de cosseno entre os próprios cursos recomendados em cada lista, por meio da função `cosine_similarity()` aplicada à submatriz TF-IDF dos vizinhos. A Qualidade das recomendações foi calculada comparando a nota média (rating) dos cursos recomendados com a nota média global do dataset.

Os resultados obtidos foram: cobertura de ~82%, similaridade média de 71,8%, diversidade média de 58,4% e nota recomendada de 4,71 frente à nota global de 4,65, indicando que o modelo tende a sugerir cursos de qualidade igual ou superior à média do catálogo.

3.9 OTIMIZAÇÃO E AJUSTES

A otimização dos hiperparâmetros foi conduzida de forma analítica, com base na compreensão do impacto de cada parâmetro sobre o desempenho do sistema, em vez de busca exaustiva por grid search, estratégia inviável na ausência de um conjunto de validação com rótulos supervisionados.

Para o `TfidfVectorizer`, o parâmetro `max_features=5000` foi definido como ponto de equilíbrio entre expressividade do vocabulário e controle de dimensionalidade. Valores menores (ex.: 1.000) reduziram a capacidade de discriminação entre cursos com habilidades distintas; valores maiores (ex.: 20.000) aumentariam o ruído de termos raros sem ganho proporcional em precisão. O `ngram_range=(1,2)` foi escolhido para capturar expressões técnicas compostas: como "machine learning", "data science" e "deep learning", que perderiam significado se fragmentadas em unigramas. O `min_df=2` elimina termos únicos (hapax legomena) que não contribuem para a similaridade entre cursos distintos.

Para o modelo KNN, o valor $K=10$ foi escolhido como padrão alinhado à literatura de sistemas de recomendação, que tipicamente adota listas de 5 a 20 itens. Testes com $K=5$ produziram listas muito curtas com alta similaridade mas baixa diversidade; $K=20$ aumentou a diversidade mas introduziu cursos com similaridade abaixo de 50%, comprometendo a relevância. O valor $K=10$ oferece o melhor equilíbrio entre os dois critérios nas métricas calculadas.

3.10 IMPLANTAÇÃO E MONITORAMENTO

O sistema de recomendação desenvolvido neste projeto foi implementado como um notebook Python (formato .ipynb), adequado para fins acadêmicos e demonstração do pipeline. Para uma implantação em ambiente de produção, seriam necessárias adaptações arquitetônicas e operacionais descritas a seguir.

Em termos de implantação, o pipeline TF-IDF + KNN pode ser serializado com a biblioteca joblib (joblib.dump/joblib.load), permitindo que o modelo treinado seja persistido em disco e carregado sem necessidade de reprocessamento a cada requisição. A função de recomendação poderia ser encapsulada em uma API REST utilizando frameworks como FastAPI ou Flask, expondo endpoints como POST /recommend com parâmetros de entrada (nome do curso ou skill de interesse, número de recomendações) e retorno em formato JSON. O artefato de software está disponível no repositório GitHub do projeto: <https://github.com/canasep/mackprojeto3>.

Em termos de monitoramento, as métricas de cobertura, similaridade e diversidade deveriam ser recalculadas periodicamente após atualizações do catálogo de cursos, detectando possível degradação do desempenho causada por drift do vocabulário, situação em que novos cursos utilizam terminologia não presente no vocabulário TF-IDF original. Nesse caso, seria necessário retreinar o vetorizador e o modelo KNN com o catálogo atualizado. Adicionalmente, o registro de consultas realizadas permitiria análise de padrões de uso e identificação de habilidades buscadas com frequência que não estejam bem representadas no catálogo.

4. REFERÊNCIAS

COVER, T. M.; HART, P. E. Nearest neighbor pattern classification. IEEE Transactions on Information Theory, v. 13, n. 1, p. 21–27, 1967. DOI: <https://doi.org/10.1109/TIT.1967.1053964>

HERLOCKER, J. L. et al. Evaluating collaborative filtering recommender systems. ACM Transactions on Information Systems, v. 22, n. 1, p. 5–53, 2004. DOI: <https://doi.org/10.1145/963770.963772>

KAPOOR, K. Coursera Courses Dataset 2021. Kaggle, 2021. Disponível em: <https://www.kaggle.com/datasets/khusheekapoor/coursera-courses-dataset-2021>. Acesso em: mar. 2026.

LOPS, P.; DE GEMMIS, M.; SEMERARO, G. Content-based Recommender Systems: State of the Art and Trends. In: RICCI, F. et al. (org.). Recommender Systems Handbook. Boston: Springer, 2011. p. 73–105.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. Introduction to Information Retrieval. Cambridge: Cambridge University Press, 2008.

PAZZANI, M. J.; BILLSUS, D. Content-Based Recommendation Systems. In: BRUSILOVSKY, P.; KOBASA, A.; NEJDL, W. (org.). The Adaptive Web. Berlin: Springer, 2007. p. 325–341.

RICCI, F.; ROKACH, L.; SHAPIRA, B. Introduction to Recommender Systems Handbook. In: RICCI, F. et al. (org.). Recommender Systems Handbook. Boston: Springer, 2011. p. 1–35. DOI: https://doi.org/10.1007/978-0-387-85820-3_1

SALTON, G.; BUCKLEY, C. Term-weighting approaches in automatic text retrieval. Information Processing & Management, v. 24, n. 5, p. 513–523, 1988.

SCIKIT-LEARN DEVELOPERS. Scikit-learn: Machine Learning in Python. Disponível em: <https://scikit-learn.org>. Acesso em: mar. 2026.

SINGHAL, A. Modern information retrieval: a brief overview. IEEE Data Engineering Bulletin, v. 24, n. 4, p. 35–43, 2001.